PipeRewind: A Time-Travel Debugger for UNIX Shell Pipelines

Systems Programming Architecture Report

Luiz Takahashi & Ehzoc Chavez
Washington State University

Abstract

UNIX shell pipelines rely on anonymous inter-process communication (IPC) channels, traditionally rendering intra-pipeline data transmission unobservable without pervasive source code modification or system call tracing. This paper introduces PipeRewind, a deterministic execution logger and time-travel debugger that seamlessly interposes on process pipelines. By utilizing `dup2` file descriptor redirection in conjunction with an `epoll`-driven asynchronous capture loop, PipeRewind multiplexes and serializes standard I/O streams into an indexed binary trace sequence. The system introduces an `ncurses`-based interactive visualization environment equipped with a sliding-window data cache and real-time process state polling via the `/proc` pseudo-filesystem, enabling high-fidelity replay of process executions without exhausting memory bounds.

1 Introduction

The modularity of the UNIX pipeline (`|`) abstraction allows developers to compose complex text-processing operations from minimal, specialized utilities. However, the standard behavior of linking standard output (`STDOUT_FILENO`) of process P_i to standard input (`STDIN_FILENO`) of process P_{i+1} via contiguous kernel pipe buffers results in extreme operational opacity. If a data corruption or failure anomaly occurs within an intermediate pipeline stage, debugging necessitates ad-hoc injection of `tee` commands or reliance on high-overhead diagnostic utilities such as `strace`, which injects significant noise via kernel context boundary intersections.

PipeRewind addresses this deficiency by establishing a transparent virtualization layer atop the standard pipeline semantic model. Rather than direct process-to-process bridging, PipeRewind acts as a multiplexed data broker. It intercepts arbitrary textual byte streams across N executing processes, serializes exact temporal events via POSIX high-resolution clocks, and persists the payload into an actionable `.prt` binary artifact for deferred or real-time inspection.

2 System Architecture and Process Topology

The instantiation of the PipeRewind trace involves establishing a rigorous inter-process topology orchestrated by a primary Capture Engine parent process.

2.1 Process Hierarchy & FD Redirection

Figure 1 illustrates the `fork()` and `exec()` lifecycle deployed for dynamic command execution. For an N-stage pipeline, the Capture Engine provisions $2(N-1)+1$ anonymous pipe file descriptors.

Prior to executing the binary payload via `execvp()`, the child process environment undergoes descriptor redirection. For pipeline stage P_i, the child invokes `dup2()` to aggressively overwrite `STDIN_FILENO` with the read-end of the inbound capture pipe, and `STDOUT_FILENO`

with the write-end of the outbound capture pipe. The child's residual file descriptors are rigorously closed to prevent internal descriptor leaking or inadvertent blocking architectures.

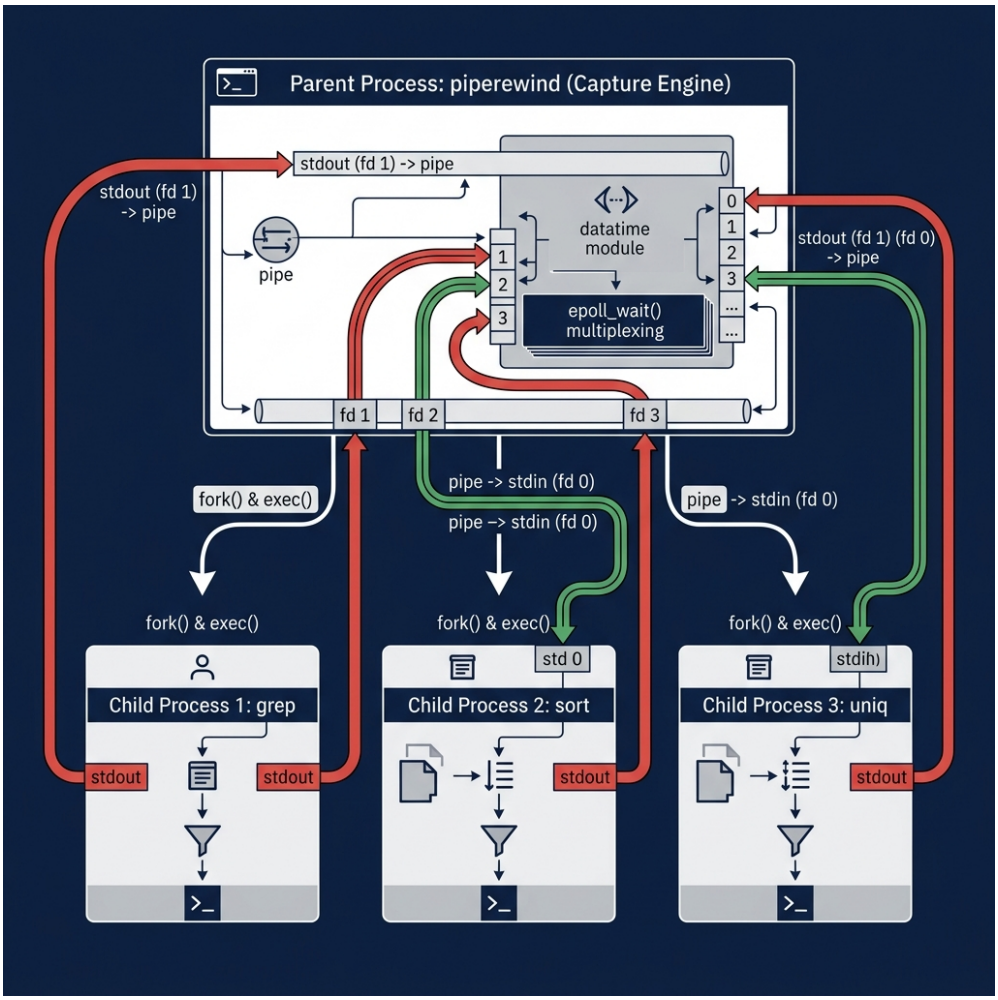

Figure 1: Process hierarchy and file descriptor abstraction. The Capture Engine interfaces via bridged pipes rather than allowing direct kernel IPC between children.

2.2 Asynchronous I/O via `epoll`

Linearly polling or blocking on individual pipe `read()` operations would inherently deadlock complex execution trees due to kernel pipe capacity limitations (64 KB typically) and divergent process scheduling behavior.

Consequently, PipeRewind leverages the horizontal trigger mechanisms of `epoll`. The Capture Engine registers the read-ends of all P_i output pipes into the epoll instance. Adhering strictly to non-blocking I/O paradigms (`O_NONBLOCK`), the system enters an `epoll_wait()` multiplexing loop. When data is flushed by any process P_i, the engine is awakened, drains the descriptor buffer into user-space, tags it with a monotonic nanosecond timestamp, and propagates the payload to the P_{i+1} input pipe.

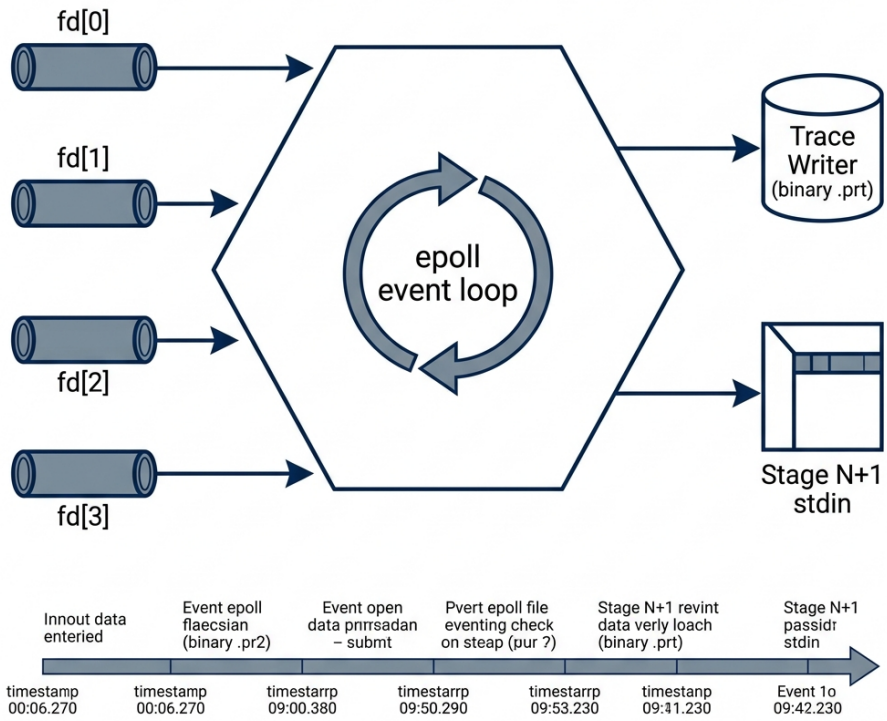

Figure 2: Event-driven architecture: The `epoll` event loop concurrently manages inbound buffers (fd_n), forwarding data structurally while serializing execution state into the Binary Trace Writer.

3 Data Persistence and Deterministic Representation

To facilitate accurate 'time-travel' replay capabilities without re-executing transient side-effecting binary trees, the sequence is serialized into the `.prt` binary schema.

A `.prt` trace minimizes serialization overhead by operating fundamentally upon binary structs written contiguously via `fwrite()`. The layout consists of:

1. **File Header**: Global topology definitions and endian-verification magic numbers.
2. **Stage Directories**: Process Command-Line Arguments, PID metrics, and associated metadata strings.
3. **Event Stream**: Interleaved, chronologically-ordered polymorphic blocks representing kernel-state changes.
4. **Payload Indexing Lookup**: A terminal indexing table providing `fseek()` localized byte-offsets mapping directly to individual sequence indices T_i, bypassing $O(N)$ sequential parses on massive filesides.

Recorded states include `PROC_START`, `PROC_EXIT`, `PIPE_EOF`, `DATA_IN`, and `DATA_OUT`.

4 Interactive Replay and Sliding-Window Caching

The core feature of PipeRewind is the `ncurses`-powered `tui.c` terminal viewer. Traditional text-visualizers succumb to out-of-memory (OOM) faults when ingesting extensive pipeline captures.

PipeRewind circumvents heap explosion through the implementation of a constrained **sliding-window caching** heuristic. While the `.prt` artifact is mapped structurally, payloads are actively `calloc`ed mapping solely bounded subsets consisting of a localized parameter array (defaulting strictly to $20,000$ adjacent sequential transactions).

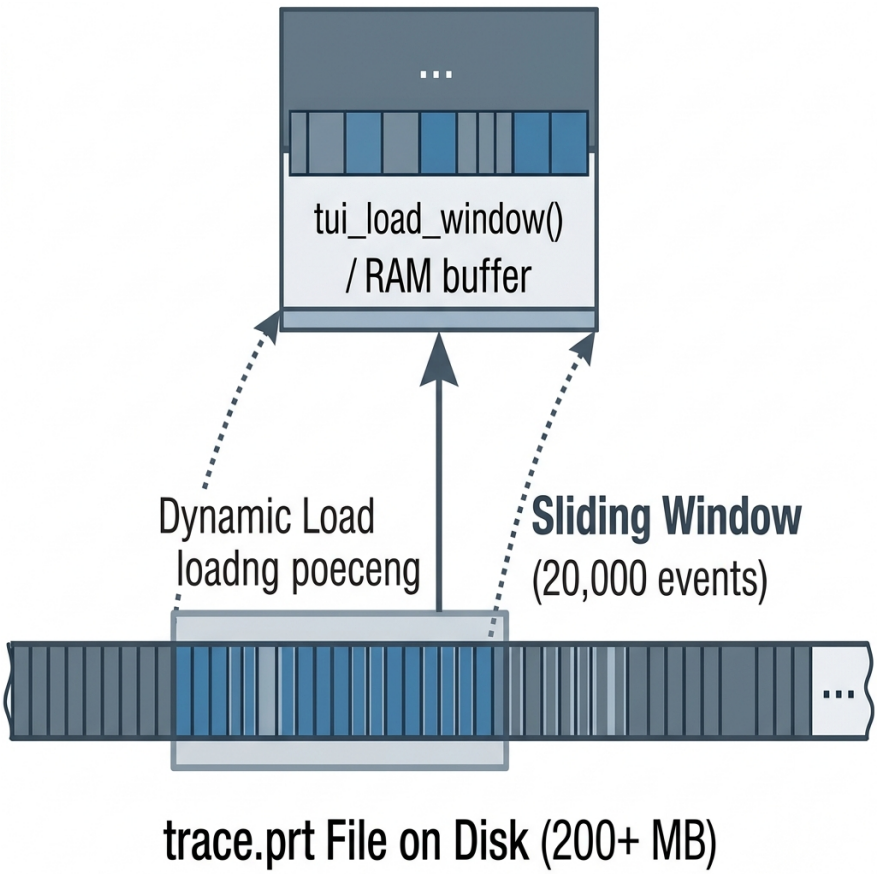

Figure 3: Sliding window trace access model: Raw `.prt` disk payloads are dynamically hoisted into memory sequentially as required by TUI navigational directives.

As navigational index pointers surpass the localized RAM boundary restrictions, dynamic destruction sequence vectors automatically `free()` stale heuristic events and recursively perform localized chunk loading.

5 Live Attachment Methodology

Advancing beyond postmortem trace evaluation, PipeRewind integrates a `live` daemon protocol. The core Capture Engine `epoll` abstraction is wrapped inside an orthogonal detached `pthread`, rendering raw events directly to the `trace.prt` handler buffer securely. Simultaneously, the TUI loop spins up on the prime directive mapping dynamic file expansion events efficiently through `trace_reader_refresh()` array reallocation logic, bridging tem-

poral pipeline monitoring securely inside execution context frameworks with asynchronous synchronization parameters.

6 Conclusion

PipeRewind establishes a formidable enhancement over standard UNIX pipeline debugging architectures. By strictly adhering to event-driven C99 design paradigms, mastering file descriptor redirection via POSIX specifications, and employing dynamic index-constrained UI paradigms, it grants engineers granular visibility into asynchronous multiprocess text streams without incurring untenable memory faults or altering underlying kernel structures.

Repository URL: https://github.com/balloonjazz/Pipe-Rewind